

Standalone Boot Mode — UART Bootloader

This guide shows how to program your firmware into the board over a single USB cable. The application is stored in QSPI flash and runs automatically at every power-on, exactly like a commercial MCU. **Uploading needs nothing but Python** — the Vitis toolchain is only involved in *building* the program (§3).

The whole loop, once you have an `.elf` (the compiled program file the build produces):

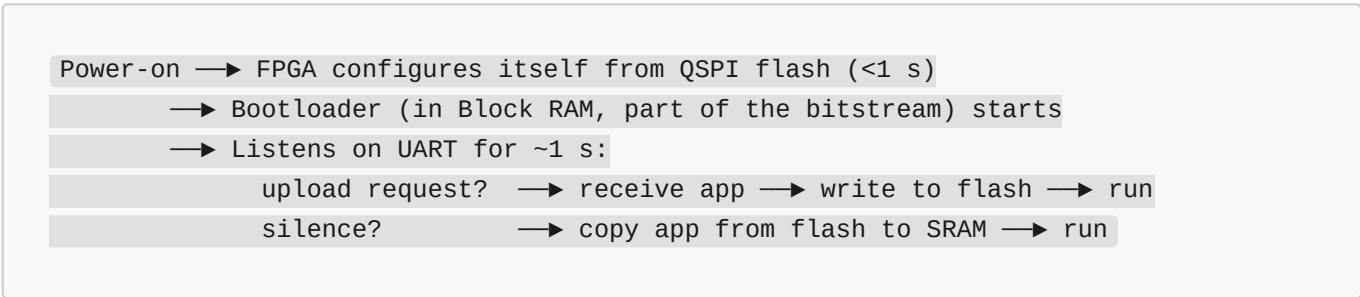
```
python3 tools/upload.py workspace-example/<app>/build/<app>.elf --monitor
```

press the reset button when asked, and watch your program print. That's it.

Prerequisites

- **Run all commands from the repository root** (`cd RISC-V-MCU-on-Cmod-A7`) — every path in this guide is relative to it.
- One micro-USB cable to the board. It carries power, the serial port, and JTAG all at once — there is no separate power supply.
- Know the two push-buttons: **BTN0 = reset**, **BTN1 = user button** (labels are printed on the board; see the pinout figure in the Pin Specification).
- A board that has been through the **one-time setup** (§2) — lab boards are usually pre-programmed. Health check: hold BTN1 while plugging in → LED0 lights. If nothing lights, the board still needs §2 (requires Vivado — ask your instructor, or do §2 yourself).
- Python 3 with `pyserial`: `sudo apt install python3-serial` — or `python3 -m pip install -user pyserial` ; if pip refuses with *externally-managed-environment*, use the apt package.
- If the port gives `PermissionError: /dev/ttyUSB...` , add yourself to its group once: `sudo usermod -aG dialout $USER` , then log out and back in.
- An application ELF built from the `app_template` (§3)

1. How Standalone Boot Works



Memory roles:

Memory	Size	Role
--------	------	------

Memory	Size	Role
QSPI flash	4 MB	bitstream (2.1 MB) + application slot @ <code>0x3000000</code> (non-volatile)
SRAM @ <code>0x60000000</code>	512 KB	where your application executes (code/data/heap, I/D-cached)
BRAM @ <code>0x00000000</code>	128 KB	32 KB bootloader (untouchable) · 32 KB ITCM @ <code>0x8000</code> (your fast code) · 64 KB DTCM @ <code>0x10000</code> (stack + fast data)

Note: The bootloader is part of the bitstream, so it is restored from flash at every power-on — like a mask ROM, it cannot be bricked from software. Holding the on-board user button (BTN1) while plugging in USB forces the board to stay in the bootloader.

2. One-Time Board Setup (Instructor)

Program the deployment image `release/boot.mcs` (bitstream + bootloader) into the QSPI flash:

1. Open Vivado **Hardware Manager > Open Target > Auto Connect**.
2. Right-click `xc7a35t_0` > **Add Configuration Memory Device** — pick the part matching the IC3 chip: `mx25l3273f-spi-x1_x2_x4` (Vivado's catalog entry covering the board's Macronix MX25L3233F — verified working) or `n25q32-3.3v-spi-x1_x2_x4` (Micron, older boards).
3. Right-click the memory device > **Program Configuration Memory Device**, select `release/boot.mcs`, keep **Erase / Program / Verify** checked, **OK**.
4. Power-cycle. LED0 lights (bootloader alive), then blinks slowly (no app yet).

Students never repeat this step; from here on the board is a pure MCU.

Tip: `release/boot.bit` is the same bitstream+bootloader as a JTAG image. In Hardware Manager (or `xsdbs + fpga release/boot.bit`) it acts as a *remote power-cycle*: the board reboots into the bootloader without touching the flash — handy for automated tests and for recovering a board whose USB you cannot reach.

3. Build Your Application

Fast path — one command (creates the Vitis project, applies the template, builds):

```
python3 tools/vitis_new_app.py myapp
# -> workspace-example/myapp/build/myapp.elf
```

Manual path: copy `main.c` and `lscript.ld` from `workspace-example/app_template/src/` into a Vitis application as in the JTAG guide (its sections 1–5; substitute the template files at its step 4.1). Either way you end up with a normal Vitis project you can open in the GUI.

The template's linker script places:

- `.text` / `.rodata` / `.data` / `.bss` / heap → **SRAM** (512 KB, cached)
- stack → top of **DTCM** (64 KB BRAM @ `0x10000`; 1-cycle, never collides with the bootloader)
- functions tagged `ITCM_FUNC` → **ITCM** (32 KB BRAM @ `0x8000`); variables tagged `DTCM_DATA` → DTCM

ITCM code ships inside the SRAM image and is copied out by `mcu_init()` at the top of `main()` — the same startup-copy idiom an STM32H7 uses for its ITCM. That is the template's **one rule**: `mcu_init();` **stays the first line of `main()`** (it also sets the UART to 115200). Put interrupt handlers and timing-critical loops in ITCM: TCM never misses, so worst-case latency equals best-case.

The template prints a memory tour at boot, so you can see the hierarchy:

```
RISC-V MCU on Cmod A7 - memory tour
main()      @ 0x600009D0  (SRAM,  cached)
blink_step() @ 0x00008000  (ITCM,  1-cycle)
blink_count @ 0x00010000  (DTCM,  1-cycle)
stack       @ 0x0001FFC0  (DTCM,  grows down)
```

Size check: `upload.py` prints `image N B` when it runs — N must stay under 524288 (512 KB). You rarely need to watch it: if the program were too big, the build would already have failed with `region 'sram' overflowed`.

Note: The same ELF works unchanged in **both** modes — Vitis **Run/Debug** over JTAG during development, `upload.py` for deployment. No rebuild, no reconfiguration.

4. Upload

```
python3 tools/upload.py workspace-example/myapp/build/myapp.elf --monitor #
flash + run + watch
python3 tools/upload.py workspace-example/myapp/build/myapp.elf --ram      # test
run from RAM (flash untouched)
```

When the script says it is **syncing**, press the **reset button (BTN0)** — the bootloader listens during its 1-second boot window after every reset. Do **not** unplug the cable while the script is running (the port would vanish from under it). If the board isn't plugged in yet: hold **BTN1**, plug in, *then* start the script — holding BTN1 makes the bootloader wait forever. A typical upload takes a few seconds (chunked transfer with CRC32 verification, then sector-erase + page-program + read-back verify).

✓ **Checkpoint** — a successful flash upload looks like this:

```
ELF: entry=0x60000000, image 8736 B @0x60000000
syncing – press the reset button (BTN0) now; or hold BTN1 while plugging in...
8736/8736 bytes (100%)
programmed to flash – app now runs at every power-on
```

```
--- UART monitor (Ctrl-C to stop watching; app keeps running)
RISC-V MCU on Cmod A7 - memory tour
...
```

Useful flags: `--monitor` to watch the app's output right away, `--port /dev/ttyUSBx` if auto-detection picks the wrong port, `--entry` for raw `.bin` files.

5. Boot and Verify

- **Power-cycle** → the board boots your flashed app automatically. No PC needed.
- `xil_printf` (the SDK's lightweight `printf`) output arrives on the same USB cable at **115200 baud** (the template sets this; the bootloader uses the same rate, so one terminal session covers both).
- To watch the output at any time, open a serial terminal: `python3 -m serial.tools.miniterm /dev/ttyUSB1 115200` (ships with pyserial; quit with Ctrl-J). The board shows up as two ports — the UART is usually the higher-numbered one.
- Convention: light an LED first thing in `main()` so "configured, booted, and running" is visible at a glance — the template's blinking LEDs are exactly this.

Note: The reset button (BTN0) restarts the CPU into the **bootloader** (BRAM is intact), which reloads your app from flash — so reset $\approx$ power-cycle in this design. Modified `.data` globals are restored because the image is re-copied from flash each boot.

6. Working with JTAG Debug Mode

Both modes coexist on the same board with no switching:

	JTAG (Vitis Run/Debug)	Standalone (<code>upload.py</code>)
Code goes to	RAM directly (volatile)	Flash (persistent)
After power-cycle	back to the flashed app	your app boots
Breakpoints / stepping	yes	no
Needs	Vivado/Vitis installed	Python + USB cable

Daily development loop: **edit** → **build** → **JTAG Run/Debug** (or `python3 tools/jtag_run.py <elf>` from a terminal); when it works, `upload.py` once to "ship" it. JTAG always has priority — it halts the CPU before the bootloader runs, so nothing in flash can interfere with a debug session.

7. How the Deployment Image Is Made (Instructor Reference)

`release/boot.mcs` = bitstream with the bootloader merged into its BRAM initialization:

```
# 1. build workspace-example/bootloader/ in Vitis
# (its checked-in lscript.ld confines it to the lower 32 KB of BRAM)
```

```
# 2. merge the ELF into the bitstream BRAM init:
tools/make_boot_mcs.sh bootloader.elf --hw release/top_wrapper/
# (wraps: updatemem -proc top_i/microblaze_riscv_0 + write_cfgmem -interface
SPIx4)
```

The same `updatemem` merge with `-out` produces `release/boot.bit` (the JTAG-loadable variant used as a remote power-cycle in §2). The bootloader source (`workspace-example/bootloader/src/bootloader.c`, ~300 lines) is deliberately readable — it doubles as course material for: reset vectors, loaders, UART protocols, SPI flash command sequences (WREN/erase/program/status-poll), CRC verification, and `fence.i`.

8. Troubleshooting

1. **upload.py can't sync** — The bootloader only listens for ~1 s after each reset: start `upload.py` *first*, then press the reset button (BTN0) when it says "syncing" — never unplug the cable mid-script. Or hold the user button (BTN1) while plugging in — that forces the bootloader to listen forever. Check the port: the board exposes two serial ports — the UART is usually the higher-numbered (`ls /dev/ttyUSB*`; select with `--port`). Close any open serial terminal first (the port is exclusive).
2. **PermissionError: /dev/ttyUSB...** — your user isn't in the port's group: `sudo usermod -aG dialout $USER`, then log out and back in.
3. **Upload OK but app misbehaves** — Ensure the app was built with the template's `lscript.ld` (sections must live in SRAM; a default all-BRAM ELF uploads but its addresses collide with the bootloader). `upload.py` warns about segments outside SRAM.
4. **No memory tour / garbage addresses** — `mcu_init()` is not the first line of `main()`, or it was removed: nothing tagged `ITCM_FUNC` / `DTCM_DATA` works before it runs.
5. **Board boots an old app** — The upload failed before the `G` step, or you used `--ram` (volatile). Re-run without `--ram` and watch for "programmed to flash".
6. **Nothing at all (no LED)** — FPGA didn't configure: flash image invalid → redo §2. JTAG always works for recovery (`tools/jtag_run.py <elf> --bit release/top.bit`).
7. **Garbage on the terminal** — Baud must be **115200** (both bootloader and template), not 9600.
8. **Want a factory-blank board** — Hardware Manager > right-click memory device > **Erase**: the FPGA then waits for JTAG at power-on.